Jetson device skills tutorial

1. Description of the function

`jetson-device-skills` is the collection of the Jetson device side Agent Skills provided by NVIDIA. It makes OpenClaw carry out the device diagnostics, the memory audits, the inference service configuration, the video capability checks and the performance tests by following the Jetson specific flows.

This article takes the ordinary OpenClaw Gateway as an example; it does not install NemoClaw and it does not use the NemoClaw sandbox.

Official repository:

```
https://github.com/NVIDIA-AI-IOT/jetson-device-skills
```

2. Get the repository

> It is already installed on the factory system, so you can skip this step.

Run the following in the Jetson terminal:

```
cd
git clone https://github.com/NVIDIA-AI-IOT/jetson-device-skills.git
cd ~/jetson-device-skills
```

3. Description of the skills and of the official installation script

The skills that are included:

- `jetson-diagnostic` captures a read only snapshot of the health of the device, including the Jetson identity, the memory, the GPU, the temperature, the power consumption, the storage, the services and the top processes.
- `jetson-memory-audit` measures the DRAM/NvMap usage and uses the real time audit data to verify the situation before and after the memory is reclaimed.
- `jetson-headless-mode` helps to turn the Jetson into a headless edge node by disabling the desktop and the secure background services.
- `jetson-inference-mem-tune` recommends the runtime and the memory flags for vLLM, SGLang, llama.cpp and TensorRT Edge-LLM.
- `jetson-llm-serve` provides the serving recipes for vLLM and SGLang that are suitable for the Jetson.
- `jetson-llm-benchmark` outputs the structured benchmark metrics for vLLM, llama.cpp and the `Ollama` path.
- `jetson-package` guides the choice of the packages, the wheels and the containers that are specific to the Jetson.
- `jetson-speculative-decoding` adds the Jetson specific EAGLE-3 or the draft model speculative decoding guidance for vLLM.

- `jetson-video-setup` installs and independently verifies the native NVIDIA video codec SDK and the PyNvVideoCodec interface.
- `jetson-video-capability` reconciles the real time NVENC/NVDEC queries and operations with the documented supported products.
- `jetson-video-recipe` turns the codec use cases into the verified, media free native and PyNvVideoCodec configuration plans.
- `jetson-video-benchmark` measures the encoding and the decoding performance of the official samples on the target device.
- `jetson-video-pipeline` performs and verifies the operations of the codec stages of the official samples and the passing of their artefacts.

Install the skills in the agent skills directory on the Jetson:

```
./install.sh
```

By default, `install.sh` links the skills into the user level skill roots of Claude Code, Codex and Cursor.

- `~/.claude/skills`
- `~/.codex/skills`
- `~/.agents/skills`
- `~/.cursor/skills`

  You can select specific targets, or copy the files instead of creating the symlinks:

```
./install.sh --targets claude,cursor
./install.sh --targets cursor-project --project /path/to/project
./install.sh --copy
./install.sh --force
```

For the NemoClaw/OpenClaw sandbox:

```
./install.sh --targets nemoclaw --nemoclaw-sandbox jetson-skills
```

Restart the agent session after the installation so that the new skill entries are loaded. If your agent expects a different skills directory, copy or sync the `skills/` directory to that location. Keep each skill as a complete directory that contains its `SKILL.md`, `scripts/` and `references/` content.

4. Connecting it to the ordinary OpenClaw

The Skills directory of the current OpenClaw workspace:

```
/home/jetson/.openclaw/workspace/skills
```

This system only sets up the openclaw environment and does not set up the Nemoclaw sandbox; the commands that create the symlinks one by one are as follows:

```
cd ~/jetson-device-skills

for s in ~/jetson-device-skills/skills/jetson-*; do n="${s##*/}";
t="$HOME/.openclaw/workspace/skills/$n"; [ -e "$t" ] || [ -L "$t" ] || { ln -s "$s" "$t";
echo "Linked: $n"; }; done
```

```
jetson@jetson:~$ cd jetson-device-skills/
jetson@jetson:~/jetson-device-skills$ for s in ~/jetson-device-skills/skills/jetson-*; do n="${s##*/}"; t="$HOME/.
openclaw/workspace/skills/$n"; [ -e "$t" ] || [ -L "$t" ] || { ln -s "$s" "$t"; echo "已链接: $n"; }; done
已链接: jetson-diagnostic
已链接: jetson-headless-mode
已链接: jetson-inference-mem-tune
已链接: jetson-llm-benchmark
已链接: jetson-llm-serve
已链接: jetson-memory-audit
已链接: jetson-package
已链接: jetson-print-device-info
已链接: jetson-speculative-decoding
已链接: jetson-video-benchmark
已链接: jetson-video-capability
已链接: jetson-video-pipeline
已链接: jetson-video-recipe
已链接: jetson-video-setup
jetson@jetson:~/jetson-device-skills$
```

An example of the link relation:

```
~/.openclaw/workspace/skills/jetson-diagnostic
    -> ~/jetson-device-skills/skills/jetson-diagnostic
```

This does not copy the files and it does not create a subdirectory named `jetson-device-skills`. The source directory has to be kept.

5. Verifying the integration

Look at the symlinks:

```
ls -l ~/.openclaw/workspace/skills/jetson-*
```

```
jetson@jetson:~/jetson-device-skills$ ls -l ~/.openclaw/workspace/skills/jetson-*
lrwxrwxrwx 1 jetson jetson   58  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-diagnostic -> /home/j
etson/jetson-device-skills/skills/jetson-diagnostic
lrwxrwxrwx 1 jetson jetson   61  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-headless-mode -> /hom
e/jetson/jetson-device-skills/skills/jetson-headless-mode
lrwxrwxrwx 1 jetson jetson   66  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-inference-mem-tune ->
 /home/jetson/jetson-device-skills/skills/jetson-inference-mem-tune
lrwxrwxrwx 1 jetson jetson   61  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-llm-benchmark -> /hom
e/jetson/jetson-device-skills/skills/jetson-llm-benchmark
lrwxrwxrwx 1 jetson jetson   57  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-llm-serve -> /home/je
tson/jetson-device-skills/skills/jetson-llm-serve
lrwxrwxrwx 1 jetson jetson   60  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-memory-audit -> /home
/jetson/jetson-device-skills/skills/jetson-memory-audit
lrwxrwxrwx 1 jetson jetson   55  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-package -> /home/jets
on/jetson-device-skills/skills/jetson-package
lrwxrwxrwx 1 jetson jetson   65  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-print-device-info ->
/home/jetson/jetson-device-skills/skills/jetson-print-device-info
lrwxrwxrwx 1 jetson jetson   68  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-speculative-decoding
-> /home/jetson/jetson-device-skills/skills/jetson-speculative-decoding
lrwxrwxrwx 1 jetson jetson   63  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-video-benchmark -> /h
ome/jetson/jetson-device-skills/skills/jetson-video-benchmark
lrwxrwxrwx 1 jetson jetson   64  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-video-capability -> /
home/jetson/jetson-device-skills/skills/jetson-video-capability
lrwxrwxrwx 1 jetson jetson   62  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-video-pipeline -> /ho
me/jetson/jetson-device-skills/skills/jetson-video-pipeline
lrwxrwxrwx 1 jetson jetson   60  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-video-recipe -> /home
/jetson/jetson-device-skills/skills/jetson-video-recipe
lrwxrwxrwx 1 jetson jetson   59  9月 14 18:10 /home/jetson/.openclaw/workspace/skills/jetson-video-setup -> /home/
jetson/jetson-device-skills/skills/jetson-video-setup
```

Look at the `SKILL.md` inside the links:

```
find -L ~/.openclaw/workspace/skills -maxdepth 2 -name SKILL.md | grep '/jetson-'
```

```
jetson@jetson:~/jetson-device-skills$ find -L ~/.openclaw/workspace/skills -maxdepth 2 -name SKILL.md | grep '/jet
son-'
/home/jetson/.openclaw/workspace/skills/jetson-video-recipe/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-llm-serve/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-video-capability/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-package/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-headless-mode/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-print-device-info/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-video-setup/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-smart-apps/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-llm-benchmark/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-inference-mem-tune/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-video-benchmark/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-video-pipeline/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-speculative-decoding/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-memory-audit/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-hardware-control/SKILL.md
/home/jetson/.openclaw/workspace/skills/jetson-diagnostic/SKILL.md
jetson@jetson:~/jetson-device-skills$
```

The `-L` is required, because the ordinary `find` does not enter the symlinked directories by default.

Restart the Gateway and start a new session:

```
openclaw gateway restart
openclaw tui
```

6. Testing some of the skills

Use the jetson-diagnostic skill to run a read only health check of the current Jetson and to summarise the device model, the system version, the memory, the GPU, the temperature, the power supply, the storage, the services and the high usage processes.

Use the jetson-diagnostic skill to perform a read-only device health check on the current Jetson, summarizing the device model, system version, memory, GPU, temperature, power, storage, services and high-load processes.

I'll search for the jetson-diagnostic skill and use it to perform the health check.## 🖥 Jetson Device Health Check

Device Identity

Field	Value
Model	NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super
SKU	orin-nano
Variant	orin-nano-8gb
Generation	Orin
L4T Version	39.2.1

Memory

Metric	Value
Total RAM	7.7 GB
Available	3.8 GB (49%)
Cached	2.9 GB
Swap Total	16 GB
Swap Used	0 MB

Temperature (°C)

Zone	Current
CPU	51.3°C
GPU	49.7°C
SoC0	50.3°C
SoC1	49.7°C
SoC2	49.4°C
Junction (Tj)	50.8°C

✅ All temperatures are within normal operating range (<85°C)

Power

Field	Value
NVPModel	unknown (not accessible)
Current Power	~5.1W (from tegrastats: VDD_IN 5136mW)

GPU Status

⚠ GPU memory source unavailable — nvidia-smi returns stub values on this nvgpu stack, and NvMap debugfs requires root access. GPU processes cannot be enumerated without elevated privileges.

Storage

Mount	Usage
/	94% used ⚠

🔒 Warning: Root filesystem is nearly full (94%). Consider cleaning up space.

🔒 Warning: Root filesystem is nearly full (94%). Consider cleaning up space.

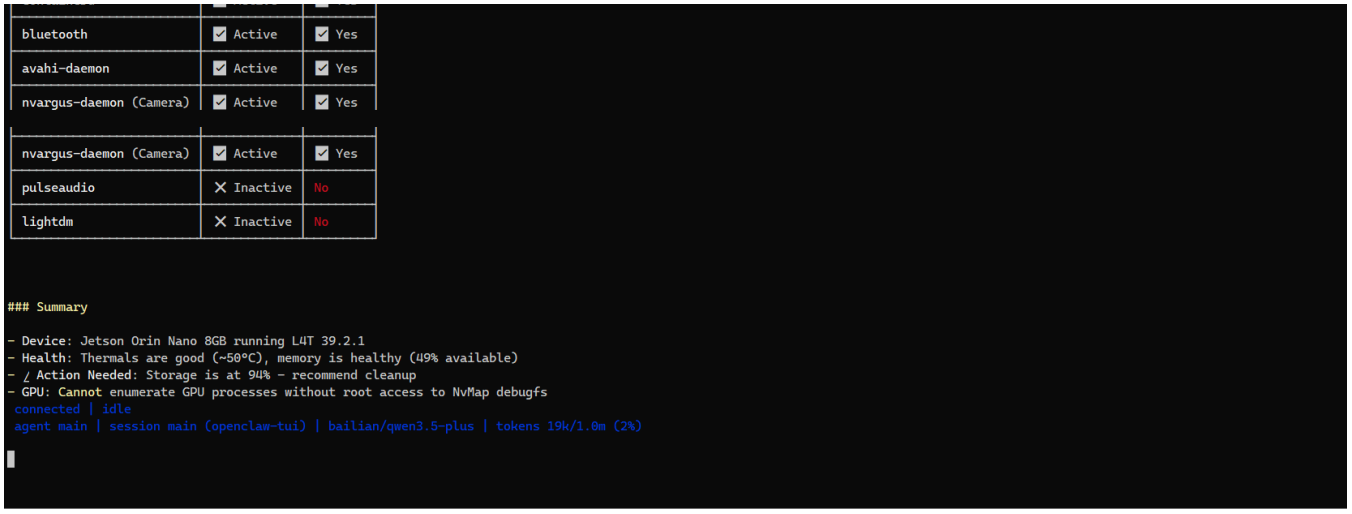
Top 5 High-Load Processes

PID	Process	PSS Memory
68621	node-MainThread	381 MB
68894	openclaw-tui	242 MB
66231	gnome-shell	231 MB
68023	update-manager	163 MB
66438	gnome-software	131 MB

Key Services Status

Service	Status	Enabled

Service	Status	Enabled
gdm3 (Display Manager)	✅ Active	alias
docker	✅ Active	✅ Yes



Use the jetson-memory-audit skill to perform a read-only check of the DRAM and NvMap usage of the current Jetson, and output the audit results.

Use the jetson-memory-audit skill to perform a read-only check of the DRAM and NvMap usage of the current Jetson, and output the audit results.

Jetson Memory Audit Report

System Identity

Field	Value
Device	NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super
SKU	orin-nano (orin-nano-8gb)
L4T Version	39.2.1

DRAM Usage Summary

Metric	Value	Percentage
Total RAM	7.73 GB	100%
Available	3.88 GB	50%
Free	0.87 GB	11%
Cached	2.93 GB	38%
Swap Total	16 GB	-
Swap Used	0 MB	0%

☑ Memory Health: 50% available, no swap pressure

tegrastats Snapshot

```
RAM 3514/7546MB (lfb 10x4MB)
SWAP 0/16384MB (cached 0MB)
CPU [6%@1728,1%@1728,2%@1728,100%@1728,8%@729,3%@729]
GR3D_FREQ 0%
```

- RAM Used: 3.5 GB / 7.5 GB (47%)
- Large Free Blocks: 10 × 4MB
- GPU Frequency: 0% (idle)

- Large Free Blocks: 10 × 4MB
- GPU Frequency: 0% (idle)

NvMap (GPU Memory) Status

⚠ NvMap debugfs not readable - Requires root access to /sys/kernel/debug/nvmap

Field	Value
Readable	false
Total Tracked	0 KB
Top Clients	(unavailable)

🔒 Note: GPU memory attribution requires elevated privileges. Run with sudo to see per-process GPU memory usage.

Top 10 Memory Consumers (by PSS)

Rank	PID	Process	PSS Memory
1	68621	node-MainThread	348 MB
2	68894	openclaw-tui	247 MB
3	66231	gnome-shell	231 MB
4	68023	update-manager	163 MB
5	66438	gnome-software	131 MB
6	2076	jupyter-lab	89 MB
7	2345	jupyter-lab	88 MB
8	66572	gnome-remote-de	88 MB
9	67059	gnome-control-c	83 MB
10	66154	gnome-remote-de	35 MB

Top 3 combined: 826 MB (11% of total RAM)

Display/Graphics Services

Service	Status	Enabled

Service	Status	Enabled
gdm3	☑ Active	alias
gdm	☑ Active	static



Use the jetson-package skill to perform a read-only check of the current JetPack/L4T, Python, CUDA and container environment, and explain which method should be preferred when installing Jetson Python packages.

Use the jetson-package skill to perform a read-only check of the current JetPack/L4T, Python, CUDA and container environment, and explain which method should be preferred when installing Jetson Python packages.

Jetson Package & Environment Report

Device Environment Summary

Component	Value
Device	NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super
SKU	orin-nano-8gb
Generation	Orin
L4T Version	39.2.1
CUDA SM Hint	8.7 (sm_87)
Python	3.12.3
pip	24.0
CUDA Toolkit	13.2 (nvcc)
Docker	29.1.3 ✅

Preferred vLLM Container Image

```
vllm/vllm-openai:latest
```

```
vllm/vllm-openai:latest
```

✅ Your device (L4T r39.2.1) qualifies for the upstream vLLM image since you're on Orin with L4T r39+.

Canonical Package Sources

Source	URL	Purpose
GHCR (NVIDIA-AI-IOT)	https://github.com/orgs/NVIDIA-AI-IOT/packages	Prebuilt Jetson containers: llama_cpp, ollama, live-vlm-webui, older Orin vllm
Jetson AI Lab PyPI	https://pypi.jetson-ai-lab.io/	Jetson-compatible Python wheels (GPU-native packages)
NGC PyTorch	https://catalog.ngc.nvidia.com/orgs/nvidia/containers/pytorch	Official PyTorch containers (verify current tag)
NGC CUDA	nvcr.io/nvidia/cuda:<ver>-devel-ubuntu<ver>	Multi-arch CUDA base containers

Preferred Package Installation Methods for Jetson

Python GPU Packages (PyTorch, onnxruntime-gpu, vLLM, etc.)

✅ PREFERRED: Jetson AI Lab PyPI Index

Generic PyPI wheels often claim aarch64 support but omit Jetson-specific SM targets (sm_87 for Orin, sm_110 for Thor). This causes installation failures or silent performance degradation.

```
# Example: Install onnxruntime-gpu for JetPack 6 / CUDA 12.6
pip install --extra-index-url https://pypi.jetson-ai-lab.io/jp6/cu126/+simple/ onnxruntime-gpu

# Set as environment variable for multiple installs
export PIP_EXTRA_INDEX_URL="https://pypi.jetson-ai-lab.io/jp6/cu126/+simple/"
pip install onnxruntime-gpu torch torchvision
```

Why this matters:

Issue	Generic PyPI Wheel	Jetson AI Lab Wheel
SM Target	May omit sm_87/sm_110	Built for Jetson GPU
CUDA Version	May mismatch L4T	Matches JetPack CUDA
CUDA Version	May mismatch L4T	Matches JetPack CUDA
Performance	Potentially slow/fallback	Optimized for Jetson
Compatibility	Risk of runtime errors	Tested on Jetson

Container Images

✅ PREFERRED: NVIDIA-Curated Sources

Use Case	Recommended Source	Example
vLLM (L4T r39+)	Upstream vllm/vllm-openai	docker pull vllm/vllm-openai:latest
vLLM (Older Orin)	NVIDIA-AI-IOT GHCR	ghcr.io/nvidia-ai-iot/vllm:latest-jetson-orin
PyTorch (Orin)	NGC with -igpu suffix	nvcr.io/nvidia/pytorch:<tag>-py3-igpu
PyTorch (Thor)	NGC main multi-arch	nvcr.io/nvidia/pytorch:<tag>-py3
CUDA Base	NGC multi-arch	nvcr.io/nvidia/cuda:12.6.0-devel-ubuntu22.04

CUDA Base	NGC multi-arch	nvcr.io/nvidia/cuda:12.6.0-devel-ubuntu22.04
LLM Tools	GHCR NVIDIA-AI-IOT	ghcr.io/nvidia-ai-iot/llama_cpp:latest

⚠ Avoid:
- Random arm64 images on Docker Hub without Jetson validation
- Legacy l4t-cuda containers (redirect to multi-arch nvcr.io/nvidia/cuda)
- Containers that don't specify SM target or JetPack version

📦 System Packages (CUDA, cuDNN, TensorRT)

✅ PREFERRED: NVIDIA APT Repository

Your system already has CUDA 13.2 installed via APT. Continue using:

Your system already has CUDA 13.2 installed via APT. Continue using:

```
# List installed CUDA packages
dpkg -l | grep cuda

# Install additional CUDA components
sudo apt update
sudo apt install cuda-toolkit-13-2 libcudnn9-cuda-13
```

🔑 Key Compatibility Rules

GPU Architecture (SM) Matters

Jetson Family	Compute Capability	Build Target	Warning
Orin (AGX/NX/Nano)	8.7	sm_87	Many desktop aarch64 wheels omit Jetson kernels
Thor (T5000/T4000)	11.0	sm_110	Requires Blackwell Jetson support

Your Device: Orin Nano (sm_87)

Your Device: Orin Nano (sm_87)

- ✅ L4T r39.2.1 → Use upstream vLLM (vllm/vllm-openai:latest)
- ✅ CUDA 13.2 → Match PyPI index to CUDA version
- ✅ Docker 29.1.3 → Container runtime ready

📋 Quick Reference: Installation Commands

PyTorch (Orin, L4T r39+)

```
# Check NGC for current tag, then:
docker pull nvcr.io/nvidia/pytorch:24.05-py3  # Verify current tag on NGC
```

onnxruntime-gpu (Jetson AI Lab)

```
pip install --extra-index-url https://pypi.jetson-ai-lab.io/jp6/cu126/+simple/ onnxruntime-gpu
```

vLLM (Your device: L4T r39+)

```
docker pull vllm/vllm-openai:latest
docker run --gpus all vllm/vllm-openai:latest --model <model-name>
```

llama.cpp (GHCR)

```
docker pull ghcr.io/nvidia-ai-iot/llama_cpp:latest
```

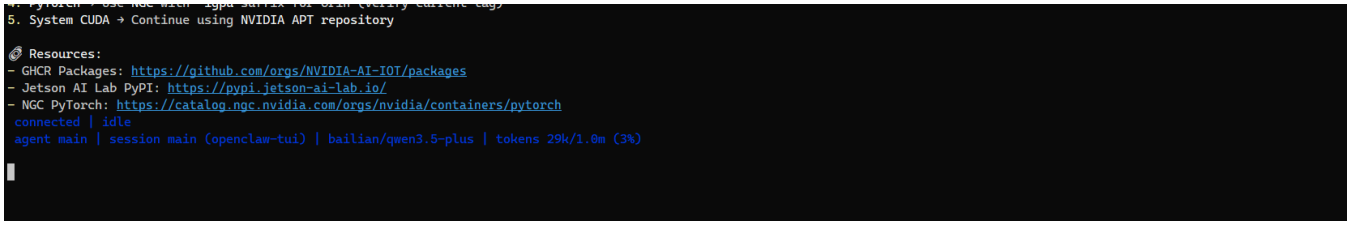
🚨 Common Pitfalls to Avoid

❌ Wrong	✅ Right
pip install onnxruntime-gpu (default PyPI)	Use Jetson AI Lab index with --extra-index-url
❌ Wrong	✅ Right
pip install onnxruntime-gpu (default PyPI)	Use Jetson AI Lab index with --extra-index-url
Random arm64 Docker images	NVIDIA-curated GHCR/NGC images
Assuming aarch64 = Jetson-compatible	Verify SM target (sm_87/sm_110)
Using l4t-cuda (legacy)	Use nvcr.io/nvidia/cuda multi-arch
Installing without checking L4T version	Run artifact_hints.sh first

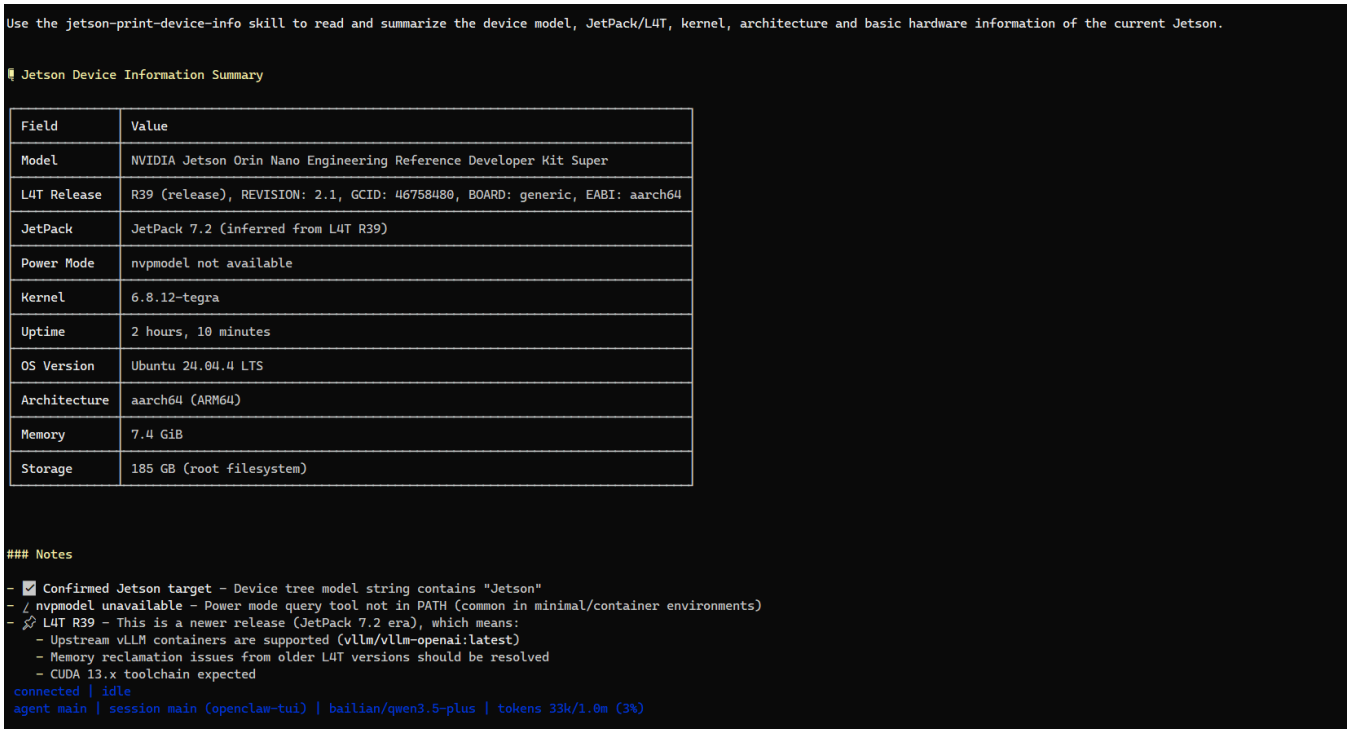
Summary

For your Jetson Orin Nano (L4T 39.2.1, CUDA 13.2, sm_87):

1. Python GPU packages → Use Jetson AI Lab PyPI with matching CUDA index
2. vLLM container → Use upstream vllm/vllm-openai:latest (L4T r39+ qualifies)
3. Other LLM tools → Use NVIDIA-AI-IOT GHCR packages



Use the jetson-print-device-info skill to read and summarize the device model, JetPack/L4T, kernel, architecture and basic hardware information of the current Jetson.



7. Rolling back the symlinks

Only delete the symlinks that point to the NVIDIA repository:

```
d="$HOME/.openclaw/workspace/skills"; for l in "$d"/jetson-*; do [ -L "$l" ] || continue;
t="$(readlink "$l")"; case "$t" in "$HOME/jetson-device-skills/skills/"*) rm "$l"; echo
"Removed: ${l##*/}";; esac; done
```

This command does not delete the source directory, and it does not delete the existing real directories `jetson-smart-apps` and `jetson-hardware-control`.

8. Points to note

- The current system does not have nemoclaw installed; only the device-skills are symlinked into the skills of openclaw.
- Do not treat `--targets codex` as an OpenClaw installation parameter.
- The source directory `jetson-device-skills` has to be kept, otherwise the symlinks stop working.
- `jetson-headless-mode` may change the state of the desktop and of the services, so you have to confirm the impact before you use it.
- The LLM and the video benchmarks may consume a lot of memory, GPU and disk space.
- The final execution rules of each Skill are the ones in its `SKILL.md` in the directory.